



INTERNATIONAL UNIVERSITY OF GRAND-BASSAM
Excellence • Accountability • Opportunity

SCHOOL OF SCIENCE AND ENGINEERING

CSC 4306 · Design & Analysis of Algorithms · Fall 2026

Why Algorithms Matter

One problem, two procedures, and a difference of a hundred million

Session 1 of 24 · Friday 18 September 2026 · 8:30 – 9:50 AM

· Prof. Guillaume Sanje Mpacko, Ph.D.

SESSION 1 OF 24

Why Algorithms Matter

Friday, 18 September 2026 · 8:30 – 9:50 AM

LEARNING OBJECTIVES

- ✓ State precisely what an algorithm is, and what distinguishes it from a program.
- ✓ Explain why two correct solutions to the same problem can differ enormously in cost.
- ✓ Read and run linear search and binary search in Python.
- ✓ Describe, in ordinary language, how the cost of each one reacts when the input grows.

Where this course is going. Every program you write is a bet: that the work it does will still be affordable when the input gets big. This course is about winning that bet on purpose rather than by luck. Today we do no mathematics at all — we simply look closely at one problem solved two ways, and let the difference speak.

1 · The Phone Book of Grand-Bassam

15 min

Imagine a printed telephone directory of Grand-Bassam: 50,000 names, sorted alphabetically. Three students are asked to find **KOUASSI Yao**.

Aïcha — flip at random

Opens a page at random, checks, tries again.
Sometimes lucky on the second flip; sometimes she is still flipping at closing time. No guarantee of ever finishing.

Bakary — read every line

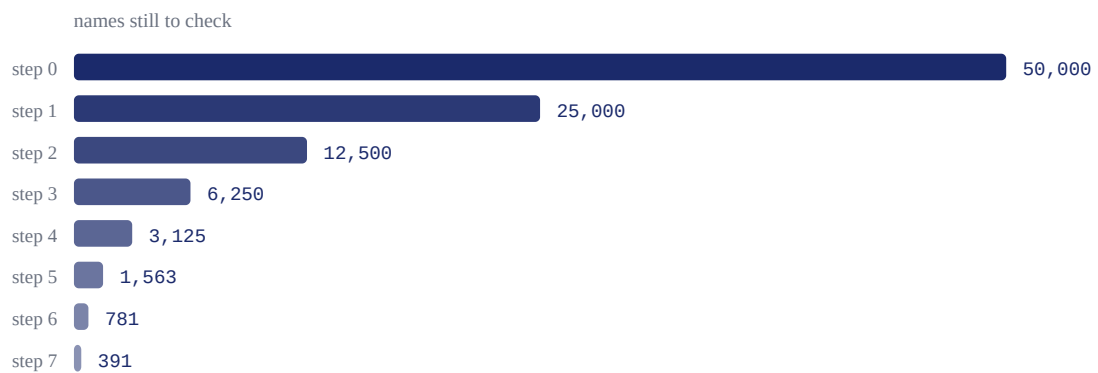
Starts at page one and reads downward.
Guaranteed to succeed, and in the worst case he reads all 50,000 entries. Reliable, and slow in a way we can predict exactly.

Chantal — halve, then halve again

Opens the middle. Is the name before or after?
Throws away half the book and repeats. She finds the name after about sixteen questions.

The point

All three have the same eyes, the same hands, the same book. The difference in performance is not in the hardware. It is in the **procedure**. That difference is the subject of this course.



... eight more halvings and only one name is left. Sixteen questions in total.

Chantal's strategy, drawn. Each question eliminates half of everything that remains. Sixteen questions suffice for 50,000 names, because doubling sixteen times from 1 passes 50,000 ($2^{16} = 65,536$). Now notice the property that matters most: **doubling the size of the phone book would cost Chantal exactly one extra question.**

SCALE IT UP BEFORE YOU MOVE ON

The Web has on the order of a hundred billion pages. Bakary's method would need a hundred billion steps — years of work. Chantal's would need about thirty-seven. Multiply that gap by the billions of searches performed every day, and you have the entire commercial value of algorithmic thinking in one comparison.

Take a moment with this

Before we go further, be sure the asymmetry has landed. Chantal is not working harder than Bakary, and she is not cleverer in the moment — each of her steps is simpler than his. She is winning because every step she takes *destroys half the remaining problem*, while every step Bakary takes destroys one name.

2 · What an Algorithm Actually Is

15 min

An **algorithm** is a finite, definite, effective procedure that transforms input into output. Five properties make that definition precise:

- 1 **Input** — zero or more values supplied from outside.
- 2 **Output** — at least one value produced as a result.
- 3 **Definiteness** — every step is precise and unambiguous. “Sort it nicely” is not a step.
- 4 **Finiteness** — it terminates after finitely many steps on *every* legal input. This is where Aïcha's random flipping fails: it might never stop.
- 5 **Effectiveness** — each step is basic enough to be carried out, in principle, with paper and pencil.

Algorithm versus program

An algorithm is the idea; a program is one incarnation of it, in one language, on one machine. Binary search is the same algorithm in Python, in C, and in Chantal's head. This distinction is what makes the course worth taking: when we prove one algorithm better than another, the conclusion survives a change of language, a change of computer, and a change of decade.

A note on notation in this course

Textbooks usually describe algorithms in pseudocode. We will not. Every algorithm in CSC 4306 is written in **runnable Python**. Pseudocode cannot be executed, cannot be tested, and cannot be timed, so it quietly hides the bugs and the costs that only appear when you run the thing. If you can write it in Python, you understood it.

3 · The Two Searches, in Python

30 min

Here are Bakary and Chantal as code. Both solve the same problem — return the index of `x` in the list `A`, or `-1` if it is absent. The second demands something extra: `A` must already be sorted.

PY LINEAR_SEARCH

```
def linear_search(A, x):
    """Bakary: check every entry, left to right."""
    for i in range(len(A)):
        if A[i] == x:
```

```
        return i
    return -1
```

Nothing here is subtle. The loop visits index 0, then 1, then 2, and returns the moment it finds a match. If it never finds one, it falls out of the loop and reports failure.

PY BINARY_SEARCH

A must be sorted

```
def binary_search(A, x):
    """Chantal: A must be sorted ascending. Halve the live range each time."""
    lo, hi = 0, len(A) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if A[mid] == x:
            return mid
        if A[mid] < x:
            lo = mid + 1           # the answer, if any, lies in the upper half
        else:
            hi = mid - 1          # the answer, if any, lies in the lower half
    return -1
```

3.1 Reading binary search slowly

This function deserves more than a glance. Three things are happening, and each one is a habit worth acquiring.

lo and hi are a promise

At every moment, the two variables delimit the only region where `x` could possibly be. Everything outside `A[lo:hi+1]` has been ruled out *permanently*.

Each test rules out half

Comparing with `A[mid]` and knowing the list is sorted eliminates an entire half in one stroke. Sortedness is what buys this; without it the comparison tells you nothing about the other elements.

The loop must shrink

Writing `lo = mid` instead of `lo = mid + 1` gives an infinite loop when `lo` and `hi` are adjacent. This is the single most common bug in the function, and you should expect to meet it.

Try it by hand

Take `A = [1, 3, 5, 7, 9, 11, 13]` and search for 11, then for 4. Write down `lo`, `hi` and `mid` at every iteration. Do this before the next section.

iteration	lo	hi	mid	A[mid]	decision
1	0	6	3	7	7 < 11 → go right, lo = 4

2

4

6

5

11

found at index 5

Searching for 11 in [1, 3, 5, 7, 9, 11, 13] — two questions for seven elements. Now do the same trace for 4, and watch the range collapse to nothing.

3.2 Running them

Type both functions into a file and check that they agree with each other. Verification by disagreement is the cheapest test you will ever write: two independent implementations of the same specification should never differ.

PY CHECKING THE TWO AGAINST EACH OTHER

```
import random

A = sorted(random.sample(range(1000), 100))    # 100 distinct sorted values

for trial in range(10_000):
    x = random.randrange(1000)                # sometimes present, sometimes not
    assert linear_search(A, x) == -1 or A[linear_search(A, x)] == x
    assert (linear_search(A, x) == -1) == (binary_search(A, x) == -1)

print("both functions agree on 10,000 random queries")
```

WHY THE ASSERTION IS WRITTEN THAT WAY

The two functions need not return the *same* index if duplicates exist, so we do not compare their return values directly. We compare what we can legitimately demand: that they agree about whether the element is present, and that any index returned really does hold `x`. Choosing the right thing to assert is a skill, and it is usually harder than writing the code.

4 • The Question This Course Is About 15 min

Both functions are correct. Both are short. So why prefer one? Let us ask the question that will occupy us for the rest of the term:

How does the work react when the input grows?

For linear search the answer is visible in the code: the loop runs once per element, so a list ten times longer takes about ten times as long. For binary search, each round discards half the remaining range, so a list *twice* as long needs just one extra round.

names in the directory	Bakary's questions (worst case)	Chantal's questions (worst case)
10	10	4
1,000	1,000	10
1,000,000	1,000,000	20
1,000,000,000	1,000,000,000	30

READ THE TABLE SIDEWAYS, NOT DOWNWARD

The interesting quantity was never “how many questions at a thousand names”. It is **how the count reacts when the size changes**. Going from ten names to a billion multiplies Bakary's work by a hundred million. It multiplies Chantal's by seven and a half.

We have no notation for this yet, and we are not going to invent one today. Next session we make the two functions count their own steps and we put a clock on them; the session after that, we give the pattern a name. The name will be a relief rather than a chore, because by then you will already have the idea it stands for.

SESSION 1 TAKEAWAY

An algorithm is a procedure, not a program; the same algorithm outlives the language and the machine. Two correct solutions to one problem can differ by a factor of a hundred million, and the difference comes from **how much of the problem each step destroys**. Bakary removes one name per step. Chantal removes half the book.

IN-CLASS EXERCISES (work in pairs)

1. Trace `binary_search([1, 3, 5, 7, 9, 11, 13], 4)` by hand. Record `lo`, `hi`, `mid` at every iteration and say exactly which test ends the loop.
2. Which of the five properties of an algorithm does Aïcha's random flipping violate? Could you repair her method so that it satisfies all five, without turning it into Bakary's?
3. Why does binary search require a sorted list? Give a concrete list and a target for which it returns the wrong answer when the list is unsorted.
4. Chantal's method needs about 16 questions for 50,000 names. About how many would she need for 100,000? For 3,200,000? Explain your reasoning in one sentence — without a calculator.

5. A recipe, a knitting pattern, and the rules of a card game: which of these are algorithms in the technical sense, and which property fails for the others?

Homework · due Tuesday 22 September

1. Type in both search functions and run the agreement check from §3.2. Submit the code and its output.
2. Deliberately introduce the `lo = mid` bug described in §3.1. What happens? Submit a short description of the failure and the smallest input that triggers it.
3. Write `binary_search_verbose(A, x)` that prints `lo`, `hi`, `mid` on every iteration. Run it on a sorted list of 1,000 elements searching for the last element, and submit the printed trace. How many lines are there?
4. In one paragraph and your own words: why does sorting the list first make searching so much cheaper, and what would you have to pay to keep the list sorted as new names arrive?
5. Reading: CLRS §1.1. Half a page on the distinction between a problem, an algorithm, and a program.

Next session · Tuesday 22 September: We stop arguing and start counting. The two functions will count their own comparisons, we will put a real clock on them, and we will meet the one diagnostic question that runs through the whole course: *what happens when I double the input?*